NovaDesk Platform Optimization Report

> **Previous filename:** `nova-desk-optimization-report.md`

Nova Desk Backend — Performance Optimization Report

From "Fails at 50 Users" to "Stable at 800 Users"

Project: Nova Desk (Express.js + MongoDB monolithic backend)
Report Type: End-to-end optimization journey (identification → fix → scale)
Period Covered: Initial load testing through 800-VU stability milestone
Status: 🟢 Stable at 800 concurrent users | 🟡 In progress toward 1000+

Table of Contents

1. Executive Summary
2. Tech Stack
3. The Starting Point
4. Methodology
5. Phase 1: Architecture-Level Fixes
6. Phase 2: Module-by-Module Debugging
7. Phase 3: Database Optimization
8. Phase 4: Caching Strategy
9. Phase 5: Horizontal Scaling
10. Load Testing Results
11. Key Learnings
12. Current Status
13. Roadmap
14. Appendix

---

1. Executive Summary

Nova Desk's backend started in a state where even **modest concurrent load (50 users) caused near-total failure** across most modules — some endpoints returned errors on nearly every request, response times ran into multiple seconds, and one module (Team) failed on **100% of requests regardless of load**, because it was a code bug rather than a scaling problem.

Through a systematic, module-by-module process — **measure → find root cause → fix → re-test → document** — the backend was brought from an unstable, largely broken state to a system that sustains **800 concurrent virtual users at a 94.57% success rate**, with all four core modules (Auth, Storage, Browser, Team) individually passing their load tests at 95–100% success rates.

The work fell into five broad categories, applied in this order:

Phase	Focus	Result
1	Architecture-level bugs (routing, middleware, health checks)	Removed multi-second latency caused by duplicate middleware execution
2	Module-specific bugs (Auth, Storage, Browser, Team)	Fixed logic errors causing 0–99% failure rates unrelated to load
3	Database optimization (indexes, connection pooling)	10x faster queries, 100x more connection capacity
4	Caching (Redis/Upstash)	70%+ reduction in response time, 80%+ cache hit rate
5	Horizontal scaling (Node.js cluster mode)	Unlocked handling of 300–800+ concurrent users by using all CPU cores

---

2. Tech Stack

Layer	Technology
Backend Framework	Express.js (monolithic, modular routers)
Database	MongoDB + Mongoose
Auth	JWT (stateless) —Access token 24h, Refresh token 7d
Password Hashing	bcrypt (reduced from 12 → 8 rounds for performance)
Caching	Upstash Redis (serverless) with in-memory fallback
Load Testing	k6, supplemented with custom PowerShell test scripts
Process Model	Node.js Cluster (multi-core worker processes)
Target Deployment	Vercel Serverless (with local dev/testing on a standalone Node server)

---

3. The Starting Point — Why It Was Failing

Before any fixes, initial smoke and stress tests exposed a system that was not gracefully degrading under load — it was **structurally broken**, independent of user count in several cases:

Module	Initial State	Notes
Auth	Partial	Core login worked, but registration took 3–6 seconds and health checks returned 401
Storage	Partial	Functionally correct but collapsed under load (7.45s P95, 24.35% failure rate)

Table 3 – continued

Module	Initial State	Notes
Browser	Broken	0.40% success rate —nearly every request failed
Team	Unstable / Broken	0% success rate under any load —a pure code bug, not a capacity issue
Portfolio	Working	4–8ms response time, no issues
Search	Working	16–19ms response time, no issues
AI / Coding	Not implemented / not tested	Routes returned 404, flagged as needing isolation before testing

A critical early realization: **not all failures were "scaling" problems.** Several modules failed at 1 virtual user just as badly as at 200 — meaning the first job wasn't to "make it faster," it was to find and fix functional bugs before performance tuning could even be meaningfully measured.

---

4. Methodology — How Issues Were Identified

The approach followed a repeatable, evidence-based loop rather than guesswork:

```
1. Run baseline load test (k6) on a single module
2. Read the failure signature (status codes, error rate, which specific check failed)
3. Trace error back through: route → middleware → controller → service → model
4. Form a hypothesis and inspect the actual code (not assumptions)
5. Apply the smallest fix that addresses the root cause
6. Re-run the exact same test
7. Compare before/after numbers
8. Document and move to the next module
```

Why this mattered: Several bugs looked like "performance" issues on the surface (e.g., "24% failure rate under load") but were actually deterministic logic bugs that happened to only be visible once concurrent traffic exposed them — for example, a missing model import causing a `500` error, or a route being commented out. Load testing was the *detection mechanism*, but the fixes were mostly **code-level, not infrastructure-level**, in the early phases.

Tools Used for Identification

- **k6** — primary load/stress testing tool; ramping-VU scenarios per module (smoke → stress → break tests)
- **PowerShell scripts** (`auth-full-check.ps1`) — deterministic functional checks (register → verify → login → protected route → token reuse)
- **Manual single-route testing** — isolating one endpoint at a time to separate "is this endpoint broken" from "is this endpoint slow"
- **Server logs / stack traces** — to catch synchronous throws (e.g., `MissingSchemaError`) that load tests alone wouldn't explain
- **`.explain("executionStats")`** in MongoDB — to confirm whether a slow query was doing a full collection scan (`COLLSCAN`) due to a missing index

5. Phase 1: Architecture-Level Fixes

These were the first fixes applied because they affected **every request across every module** — fixing them first made all subsequent module-level testing accurate (otherwise architecture bugs would mask or exaggerate module-specific issues).

5.1 Health Check Returning 401

Problem: `/api/v1/health` was defined *after* authentication middleware in `app.js`, so monitoring tools (load balancers, Kubernetes probes, uptime checkers) received `401 Unauthorized` instead of a health status.

Fix: Moved the health check route to the very top of `app.js`, before any middleware — CORS, session handling, rate limiting, and route imports.

Result: ✅ Health checks now return `200 OK` without requiring credentials.

5.2 Duplicate Route Mounting

Problem: Several modules were mounted twice — once at a specific path and again at the bare `/api/v1` path:

```
// BEFORE — broken
app.use('/api/v1', portfolioModule);     // bare path
app.use('/api/v1/teams', teamModule);
app.use('/api/v1', teamModule);          // duplicate mount
app.use('/api/v1/storage', storageModule);
app.use('/api/v1', notificationModule);  // bare path
```

Every incoming request was passing through multiple routers and their middleware chains redundantly — including authentication being re-executed multiple times — before ever reaching the intended handler. This was the single largest contributor to **10–35 second response times** observed in early tests.

Fix: Gave every module an explicit, unique mount path and removed all duplicate/bare mounts:

```
// AFTER — fixed
app.get('/api/v1/health', ...);                                    // no middleware, top of file
app.use('/api/v1/auth', ipLockout, authLimiter, authModule);
app.use('/api/v1/storage', apiLimiter(60_000, 30), storageModule);
app.use('/api/v1/teams', apiLimiter(60_000, 30), teamModule);
app.use('/api/v1/browser', apiLimiter(60_000, 30), browserModule);
app.use('/api/v1/notifications', apiLimiter(60_000, 30), notificationModule);
app.use('/api/v1', portfolioModule);   // only one bare mount remains, intentionally
```

Result: ✅ Response time for affected routes dropped from 10–35s to **under 1 second**.

5.3 Rate Limiting Returning 403 Instead of 429

Problem: Rate limiting used a single, IP-wide limiter that (a) returned the wrong HTTP status code (`403 Forbidden` instead of the standard `429 Too Many Requests`) and (b) applied IP-based lockouts that

spilled over onto authenticated users sharing an IP — meaning a failed login attempt from one user could lock out a different, already-authenticated user on the same network.

Fix: Replaced the single limiter with four purpose-specific limiters:

Limiter	Window	Limit	Key	Status Code
`globalLimiter`	1 min	100 req	IP (DDOS protection only)	429
`authLimiter`	15 min	5 req	Email	429
`apiLimiter`	1 min	20–30 req	User ID	429
`ipLockout`	15 min	5 failed attempts	IP	429

Also added `validate: { ip: false }` to satisfy `express-rate-limit` v7+'s stricter custom-key-generator validation, which was otherwise crashing the server on startup.

Result: ✅ Correct status codes returned; authenticated users no longer affected by unrelated rate-limit events.

---

6. Phase 2: Module-by-Module Debugging & Fixes

6.1 Auth Module

Before: Registration and OTP verification took **3,000–6,000ms** per request.

Root cause: Email sending via Nodemailer was `await`-ed directly in the request handler, meaning the HTTP response was blocked until the email actually left the server (an I/O-bound operation with no bearing on whether registration itself succeeded).

Fix — fire-and-forget email dispatch:

```
const sendEmailAsync = (emailFn, ...args) => {
  emailFn(...args).catch(err => console.error('Background email failed:', err));
};

// Used in registration, OTP verification, resend, and forgot-password flows
sendEmailAsync(emailService.sendOTP, email, otp, name, type);
```

Error handling was added inside the background call so that an email provider failure (e.g., SMTP timeout) could never cause the registration request itself to fail.

Result:

Metric	Before	After	Change
Registration time	3000–6000ms	150–300ms	**95% faster**
Auth success rate	100%	100%	Maintained
Health check	401	200	Fixed
Rate limiting	403	429	Fixed

6.2 Storage Module

Before: P95 response time of **7.45s**, **75.65% success rate** — roughly 1 in 4 requests failing under a 150-VU stress test.

Root causes identified:

1. Missing/insufficient compound indexes on `File` and `Folder` collections → full collection scans
2. N+1 query pattern — separate DB round-trips per file/folder instead of batched queries
3. No caching layer — every request hit MongoDB directly
4. Complex `$lookup` aggregations adding join overhead
5. Sequential (not parallel) updates for related writes (e.g., folder size + storage usage updated one after another instead of concurrently)
6. Local MongoDB instance with limited disk I/O

Fixes applied:

Indexes:

```
db.files.createIndex({ user: 1, folder: 1, isDeleted: 1, createdAt: -1 });
db.files.createIndex({ user: 1, isDeleted: 1, createdAt: -1 });
db.files.createIndex({ name: 'text' });
db.files.createIndex({ shareToken: 1, isDeleted: 1 });
db.folders.createIndex({ user: 1, parent: 1, isDeleted: 1 });
db.folders.createIndex({ user: 1, isStarred: 1, isDeleted: 1 });
```

Query optimization:

- Replaced `$lookup` aggregation with separate, targeted queries
- Added `.lean()` to all read-only queries to skip Mongoose document overhead
- Added a 10-second query timeout to prevent hanging requests
- Capped pagination at 50 items per page
- Converted sequential updates to `Promise.all()`
- Used `bulkWrite()` for cascading folder-size updates instead of walking the folder tree with individual `.save()` calls

Caching:

- Migrated from local Redis to **Upstash Redis** (serverless, lower latency)
- Cached file listings, folder contents, and storage metadata (10–30 min TTL)
- Added an in-memory cache fallback if Redis becomes unavailable

Result:

Metric	Before	After	Improvement
P95 Response	7.45s	2.28s	**70% faster**
Success Rate	75.65%	98.18%	**+22.53%**
Failure Rate	24.35%	1.82%	**−92.5%**

Table 6 – continued

Metric	Before	After	Improvement
Checks Passed	66.08%	100%	**+33.92%**

6.3 Browser Module

Before: 0.40% success rate — the module was effectively non-functional under any load.

Root causes identified:

1. `authBrowser` middleware missing on several route files, so requests weren't authenticated at all
2. Code was reading `req.user._id` in places where the browser-specific auth middleware actually attached the identity to `req.browser._id` — an object-reference mismatch causing silent failures
3. Route ordering bug: a catch-all `/:identifier` route was declared *before* specific routes like `/me` and `/followers`, so Express matched the wrong handler
4. Duplicate upload endpoint definitions
5. No caching; N+1 queries for user lookups in loops

Fixes applied:

- Added `authBrowser` middleware consistently across all browser route files
- Corrected all references from `req.user._id` to `req.browser._id`
- Reordered routes so specific paths (`/me`, `/followers`, `/following`, `/home/stats`) are declared before the catch-all `/:identifier`
- Removed the duplicate `/upload/image` route definition
- Batched user lookups (reduced from N separate queries to 2 batched queries)
- Added Redis caching for profile, followers, and following data (5–10 min TTL), with cache invalidation on follow/unfollow actions

Result (after iterative fixes — three test runs were needed as new issues surfaced):

Run	Success Rate	Notes
Run 1	74.99%	`/home/stats` and `/home/me` failing at 0%
Run 2	88.22%	`/home/stats` still failing
Run 3 (final)	**98.06%**	All 6 endpoints passing at 100%

Metric	Before	After	Improvement
P95 Response	5–8s	2.30s	**70% faster**
Success Rate	0.40%	98.06%	**+97.66%**
Failure Rate	99.60%	1.94%	**−98%**

6.4 Team Module

This module is the clearest example in the entire project of **"load testing revealed a bug, but the bug had nothing to do with load."** The `/api/v1/teams/stats` endpoint failed on **100% of requests** at 1 VU or 200 VUs alike. Root-causing it required tracing through five stacked, independent issues — fixing one revealed the next.

Issue 1 — Unregistered Mongoose models (→ `500` errors)

```
// team_Service.js
const TeamTask = mongoose.model('TeamTask');       // throws synchronously if unregistered
const TeamMessage = mongoose.model('TeamMessage');
```

`mongoose.model(name)` without a schema only succeeds if that model was already registered elsewhere via a prior import. Neither model was imported anywhere in this file's load chain, so this line threw `MissingSchemaError` *before* `Promise.all` even executed — every request landed in the controller's `catch` block and returned `500`.

Fix: Import both models directly at the top of the service file instead of relying on registration happening as a side effect elsewhere:

```
import TeamTask from '../models/team_task.model.js';
import TeamMessage from '../models/team_message_model.js';
```

Issue 2 — Inconsistent response field (`metadata` vs `data`) Every other endpoint in the codebase wrapped its payload as `data: result`; `getTeamStats` alone used `metadata: stats`. Depending on how the response wrapper class serializes the object, this could silently drop the payload from the response body.

Fix: Standardized on `data: stats` to match the rest of the codebase.

Issue 3 — Route never actually wired up The `/stats` route was commented out. The only live routes were `/:teamId/stats`, which requires two path segments. A request to `/stats` (one segment) didn't match that route, so Express fell through to `/:teamId`, treating the literal string `"stats"` as a team ID — hitting an entirely different controller and returning a `success: false` "team not found" response instead of a crash.

Fix: Uncommented the static route and placed it **before** the dynamic `/:teamId/stats` route so Express matches it first.

Issue 4 — Middleware assumed a route parameter that doesn't exist `checkTeamStatsAccess` middleware read `req.params.teamId` to check access — but `/stats` (unlike `/:teamId/stats`) has no such parameter. This meant the lookup always failed and every request was rejected with `403`, regardless of user or team.

Fix: Removed `checkTeamStatsAccess` from the `/stats` route, since the controller already resolves the requesting user's own team internally, making the middleware redundant on this specific route.

Issue 5 — Valid empty state treated as an error Once requests correctly reached the controller logic, a final issue surfaced: users with no team membership (a normal, expected state — team membership is always opt-in in this system) were receiving `404 Not Found` instead of a valid response.

Fix:

```
if (!member) {
  return new SuccessResponse({
```

```
    message: 'No team associated with this user',
    data: { memberCount: 0, taskCount: 0, channelCount: 0, messageCount: 0 },
  }).send(res);
}
```

Additional module-wide improvements (beyond the `/stats` bug):

- Added Redis caching for `getTeamById`, `getUserTeams`, `listUserTeams`, and `getTeamMembers` (5–10 min TTL)
- Added compound MongoDB indexes for `Team`, `TeamMember`, and `TeamTask` collections
- Replaced multiple sequential lookups with a single `.lean()` query where possible

Result:

Metric	Before	After	Improvement
Success Rate	0%	99.10%	**Fixed**
P95 Response	5–8s (timeouts)	1.31s	**75% faster**
Failure Rate	100%	0.90%	**−99%**
Checks Passed	0%	100%	**Fixed**

---

7. Phase 3: Database Optimization

7.1 Indexing Strategy

The rule applied consistently across all modules: **any field used in a `find()`/`findOne()` filter must be indexed**, verified using `.explain("executionStats")` to confirm the query planner isn't falling back to a `COLLSCAN` (full collection scan).

Per-module indexing guidelines used:

Module	Key Indexes
Auth	`email` (unique), `status`
Team	`team + user` compound (membership lookups), `team + status`, `team + createdAt`
Storage	`user + folder + isDeleted`, `user + isDeleted + createdAt`, `shareToken`
Search	Dedicated text index (or MongoDB Atlas Search) — regular field queries are too slow for text search

Impact: Indexed queries ran approximately **10x faster** than their un-indexed equivalents across the board.

7.2 Connection Pool Tuning

```
// Before
maxPoolSize: 10
minPoolSize: 2
```

```
// After
maxPoolSize: 1000
minPoolSize: 200
maxConnecting: 50
waitQueueTimeoutMS: 60000
```

This increase was necessary once cluster mode (Section 9) introduced multiple worker processes, each maintaining its own connection pool — the old pool size of 10 could not support concurrent load across 8 worker processes.

7.3 Read/Write Considerations

For read-heavy modules (Search, Team listings), read replicas were identified as a future scaling lever once traffic grows further — keeping write-heavy/critical paths (Auth, Storage writes) on the primary database.

---

8. Phase 4: Caching Strategy

Provider: Upstash Redis (serverless), chosen over a locally hosted Redis instance for lower operational overhead and better performance characteristics under the target deployment (Vercel serverless).

Design principles applied:

- Cache reads that are expensive and don't need to be real-time (team details, file listings, profile data)
- Always pair a cache with a documented invalidation trigger (e.g., clear team cache on member add/remove; clear browser cache on follow/unfollow)
- Fall back to an in-memory cache if Redis is unreachable, rather than failing the request

TTL strategy by module:

Module	Cache Key Pattern	TTL
Storage	`files:*`, `storage:*`, `folder:*`	10–30 min
Browser	`browser:*`, `feed:*`, `home:*`	5–10 min
Team	`team:*`, `user:teams:*`	5–10 min

Result: ~80%+ cache hit rate observed, contributing to the 70% response-time reductions seen in Storage and Browser modules.

---

9. Phase 5: Horizontal Scaling (Cluster Mode)

9.1 The Problem

Node.js runs JavaScript on a **single thread**. Regardless of how optimized the code and database were, one CPU core handling all incoming requests becomes the hard ceiling once concurrency increases. This

was confirmed directly: even after all module-level fixes, 300+ concurrent users caused the event loop to become the bottleneck, leading to timeouts.

9.2 The Solution

Implemented Node.js's built-in `cluster` module to fork one worker process per available CPU core:

```
// cluster.js
import cluster from 'cluster';
import os from 'os';

if (cluster.isPrimary) {
  const numCPUs = os.cpus().length; // 8 cores on the test machine
  for (let i = 0; i < numCPUs; i++) {
    cluster.fork();
  }
} else {
  await import('./server.js');
}
```

Each worker runs an independent instance of the full Express application and shares incoming connections via the OS-level load balancing that Node's cluster module provides.

Result: 300 VUs success rate jumped from **76.85% → 99.58%** after enabling cluster mode — this was the single highest-leverage change for raw concurrency capacity, more impactful than any individual query optimization at this scale.

---

10. Load Testing Results Timeline

10.1 Concurrency Scaling (8-core cluster, all modules combined)

VUs	Success Rate	Failure Rate	P95 Response	Status
300	99.58%	0.42%	2.69s	✅ Passed
500	98.33%	1.67%	4.19s	✅ Passed
800	**94.57%**	**5.43%**	**6.55s**	⚠️ Stable, needs further optimization
1000	34.20%	65.80%	6.66s	❌ Not yet stable

10.2 Per-Module Results at 50 VUs (final, isolated module tests)

Module	Success Rate	P95 Response
Auth	100%	< 200ms
Storage	98.18%	2.28s
Browser	98.06%	2.30s
Team	99.10%	1.31s

10.3 Per-Module Results at 500 VUs (combined load)

Module	P95 Response	Target	Status
Auth	2.35s	< 1s	⚠️ Needs work
Storage	3.70s	< 3s	⚠️ Needs work
Browser	4.69s	< 3s	⚠️ Needs work
Team	4.78s	< 3s	⚠️ Needs work

Interpretation: Each module performs excellently in isolation at low-to-moderate concurrency. The remaining gap is specifically about **combined load at high concurrency (500+ VUs across all modules simultaneously)** — this is a resource-contention problem (CPU, connection pool, memory) rather than a per-module logic problem.

---

11. Key Learnings

Lesson	Explanation
Load testing surfaces logic bugs, not just performance limits	The Team module' s 0% success rate at any load was a code bug (missing imports, wrong route, wrong middleware), not a scaling issue. Always trace failures to root cause before assuming "needs optimization."
Architecture bugs multiply everything downstream	Duplicate route mounting made every module look slower than it actually was; fixing this first was necessary for accurate module-level measurements.
Never block a response on I/O that doesn' t need to be synchronous	Email sending was the single biggest quick win in the entire project (95% latency reduction) via a simple fire-and-forget pattern.
Route order matters in Express	Catch-all routes (`/:id`) must always be declared after specific routes, or they silently intercept requests meant for other handlers.
Consistency prevents subtle bugs	Using `req.user` vs `req.browser` inconsistently, or `data` vs `metadata` inconsistently, caused failures that had nothing to do with the actual business logic.
N+1 queries are a silent tax at scale	Individually fast queries in a loop become a major bottleneck under concurrent load; batch or aggregate instead.
Indexes are the highest ROI database fix	10x query speed improvement for comparatively low effort —always the first thing to check with `.explain()`.
Vertical fixes have a ceiling; horizontal scaling breaks through it	No amount of query or caching optimization alone got past ~300 concurrent users —enabling multi-core cluster mode was what unlocked 500–800 user capacity.

Table 15 – continued

Lesson	Explanation
“No data” is not the same as “an error”	Empty states (e.g., a user with no team) should return a valid `200` response with empty/zeroed data, not a `404`.

---

12. Current Status

✅ Working Well

- Health checks: `200 OK`, no auth required
- Registration & OTP flows: 150–300ms (down from 3–6 seconds)
- Rate limiting: correct `429` status codes, scoped correctly (no spillover to authenticated routes)
- All four core modules (Auth, Storage, Browser, Team) individually pass load tests at 95–100% success
- Redis caching: 80%+ hit rate
- Cluster mode: 8 worker processes actively load-balancing
- System sustains **800 concurrent virtual users at 94.57% success rate**

⚠️ Known Limitations

Issue	Priority	Notes
1000+ VU performance	High	Success rate drops sharply (34.20%) —likely a database connection or memory ceiling
Combined-module response times at 500+ VUs	Medium	Each module is fast individually; combined resource contention increases P95 to 2.3–4.8s
Local MongoDB instance	Medium	Disk I/O on localhost is a known ceiling; cloud-hosted MongoDB (Atlas) is expected to help significantly
AI / Code execution modules	Not yet load tested	Requires isolation (separate worker/process) before testing, due to CPU-heavy or long-running I/O characteristics

---

13. Roadmap — Scaling Beyond 800 Users

Priority	Action	Expected Impact
1	Migrate from local MongoDB to MongoDB Atlas (cloud)	−2s response time, +200 VU capacity

Table 17 – continued

Priority	Action	Expected Impact
2	Increase Node.js memory allocation (e.g., to 12GB)	+100 VU capacity
3	Further optimize Team module queries	−1s response time
4	Adopt PM2 for cluster process management (in addition to raw `cluster` module)	More resilient process supervision, easier zero-downtime restarts
5	Add further indexes for Team/ Browser under combined load	−1s response time
6	Introduce read replicas for read-heavy modules (Search, Team listings)	Reduces primary DB load
7	Isolate AI/Chat and Coding-execution modules into separate worker processes or containers	Prevents CPU-heavy or long-running tasks from starving the main API event loop
8	Switch list endpoints to cursor-based pagination	Removes skip/limit performance degradation on large collections
9	Add structured monitoring (per-module dashboards)	Enables proactive detection of the *next* bottleneck instead of reactive load testing

Realistic outlook: Reaching stable 1000+ VU performance is expected to require the Atlas migration and connection/memory tuning above — the current 1000-VU failure pattern (65.80% failure, but *not* a slow P95) suggests a hard resource ceiling (likely DB connections or memory) rather than a logic bug, consistent with everything below 800 VUs having already been debugged at the code level.

---

14. Appendix: Commands & Scripts Reference

Health Check

```
Invoke-RestMethod -Method GET -Uri "http://localhost:3800/api/v1/health"
```

Get Auth Token

```
$body = @{ email = "test_0@example.com"; password = "Test@123456" } | ConvertTo-Json
$res = Invoke-RestMethod -Method POST -Uri "http://localhost:3800/api/v1/auth/login" -Body $body
↪  -ContentType "application/json"
$token = $res.data.tokens.accessToken
$headers = @{ Authorization = "Bearer $token" }
```

Run Server (with increased memory)

```
node --max-old-space-size=4096 app.js
```

k6 Test Scripts

Script	Purpose
`tests/auth/auth-smoke-test.js`	5 VU smoke test
`tests/auth/auth-registration.js`	Register + Verify + Login flow
`tests/auth/auth-stress-test.js`	High-VU stress test
`tests/auth/auth-break-test.js`	Breaking-point test
`tests/storage/storage-stress-test.js`	Storage module load test
`tests/browser/browser-stress-test.js`	Browser module load test
`tests/team/team-stress-test.js`	Team module load test
`tests/home/home-stress-test.js`	Combined-module test
`auth-full-check.ps1`	Deterministic functional check (register → verify → login → protected routes → token reuse)

```
k6 run tests/auth/auth-smoke-test.js
k6 run tests/storage/storage-stress-test.js
k6 run --vus 50 --duration 1m tests/home/home-stress-test.js
```

MongoDB Index Creation

```
use novadesk-core;

db.files.createIndex({ user: 1, folder: 1, isDeleted: 1, createdAt: -1 });
db.folders.createIndex({ user: 1, parent: 1, isDeleted: 1 });
db.teams.createIndex({ createdBy: 1, status: 1 });
db.teammembers.createIndex({ team: 1, user: 1 }, { unique: true });
db.team_tasks.createIndex({ team: 1, status: 1 });
```

Report compiled from internal testing logs, k6 output, and debugging notes across the full optimization cycle.